

RTCS

Real-time operating system dedicated to meeting specific timing constraints.

不一定快,不全是关于任务调度 (scheduling AND memory allocation)

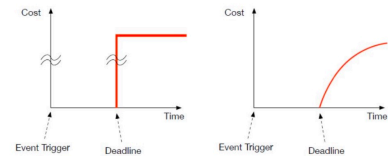
Predictability and determinism. Speed is practically important.

Hard vs soft real time

Hard real time: failure to meet the deadlines is **catastrophic**.

The timing correctness is as important as the functional correctness.

Soft real time: failure to meet the deadlines is acceptable as long as it is delivered but will **degrades** the performance.



Example:

Hard real-time tasks:

- Missile guidance systems
- Air traffic control systems
- Industrial control systems.

Soft real-time tasks:

- Video playing;
- Online gaming;
- Handling input data from the keyboard;

???

Ex: How could different tasks determine the design of your computer systems?

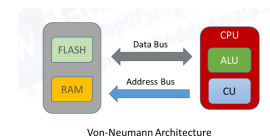
(i) Improve performance of the system; avoid wasting computing resources; provide priorities to those that are more time-critical.

(ii) Using interrupt vector table and Multithreading/concurrency.

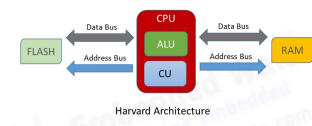
Von Newman and Harvard core processor architectures

Von Neumann architecture has one shared program and data memory on the same bus.

- Cheaper in cost.
- Speed: The CPU is not able to read/write data and access instructions at the same time. (slower)
- Control circuits: Requires lesser architecture (circuit) and space.
- Memory: Does not waste any memory space



Von-Neumann Architecture



Harvard Architecture

Harvard architecture has two separate bus for program and data.

- More expensive
- Speed: The CPU can easily read/write data as well as access the instructions at the same time.

(faster)

- Control circuits: Requires more hardware and space.
- Memory: Can result in space wastage.

note: M0 用的是 Von Neumann.

What are the main considerations when implementing them in a practical application?

The main consideration is when sharing the same bus, one must wait for the other; Considering speed, control circuits and memory.

Ex: intel x86 vs ARM

x86:

- High performance: Suitable for intensive computing tasks
- Extensive software ecosystem.
- Support complex instruction sets.
- Higher power consumption and heat production.
- High cost.

ARM:

- Lower power consumption and heat production.
- Cost efficiency: Generally cheaper to produce due to simpler design.
- Lower performance.

ARM Processor Families:

- **Cortex-A** series (Application): High performance processors for open Operating Systems;
- **Cortex-R** series (Real-time): Exceptional performance for real-time applications;
- **Cortex-M** series (Microcontroller): Cost-sensitive solutions for deterministic microcontroller applications; Energy-efficiency. Smaller code. Faster software development and reuse.

Ex: What are the rationales for developing a system-on-a-chip (SoC) for a new application? If you were asked to design a SoC for a Bluetooth headphone, which family of ARM processors would you use and why?

Power consumption, cost, area, performance, security, reliability

Cortex-M seems a good choice as the headphone requires **low power consumption, low cost, simple tasks and less functions.**

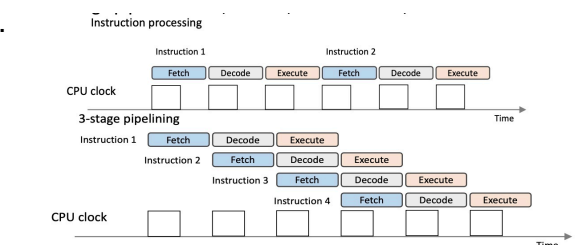
Pipeline

Pipelining is a mechanism in **RISC** processor which is used to execute instructions.

Three-stage pipeline: fetch, decode, and execution;

- Fetch: **loads** an instruction from memory.
- Decode: **identifies** the instruction to be executed.
- Execute: **processes** the instruction and **writes** the result back to a register.

Pipelining improves CPU performance by allowing **multiple** instructions to be executed



simultaneously, reducing overall processing time.

note: CISC instructions do not fit pipelined architectures very well. For pipelining to work effectively, each instruction needs to have similarities to other instructions, at least in terms of relative instruction complexity.

Cortex-M0 Processor

The smallest ARM processor

High code density

Lower power

Simplicity: Only 56 instructions, C friendly, More deterministic response time.

Uses ARMv6-M Architecture

ARM processor is developed using one of the ARM architectures;

32-bit Reduced Instruction Set Computing (RISC) processor

Von-Neumann core architecture: Both data and instructions share a single bus interface;

Registers include sixteen 32-bit registers for both general and special usages.

4 GB of memory address space

CISC vs RISC

CISC:

hardware-centric

Instruction Complexity: complex and **variable length** instructions, the task of decoding instructions is quite complex. instruction can take **several clock cycles**, require high execution time.

large number of instructions

Code Size: Compact

Development Time: Faster for complex tasks

RISC:

software-centric

Instruction Complexity: simple, **standardized instructions**, instruction decoding is simpler small number of instructions. instruction usually only takes **one clock cycle**, require less time for execution.

Code Size: Larger codebase

Development Time: Slower due to more instructions needed

Level of Abstraction:

- Architecture (Instruction Set Architecture or "ISA")
 - A set of specifications that allows developers to write software and firmware
 - These include the instruction set, exceptions, memory models.
 - ARM architecture also define **system registers**, **debug** and trace architectures and details of the **virtual memory system**
- Microarchitecture
- Hardware or Implementation

Ex:

An ARM processor is the clear choice for designing a wearable fitness product due to its:

1. Superior power efficiency for extended battery life.
2. Compact size for small wearable designs.
3. Balanced performance for fitness monitoring tasks.
4. Cost-effectiveness and ecosystem support.

Ex: List two main features of a real time computer system compared with an embedded system?

Deterministic Timing Behavior: A real-time system guarantees task execution within strict, predictable time constraints

Task Prioritization and Scheduling: A real-time system often uses specialized scheduling algorithms to handle multiple tasks with different priorities, ensuring critical tasks are completed on time.